

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage.feature import hog
from skimage import exposure
import time
from google.colab import files
uploaded = files.upload()
image_path = list(uploaded.keys())[0]
img = cv2.imread(image_path)
if img is None: print("Error: Image could not be loaded.")
else: print("Image loaded successfully:", image_path)
print("Image shape:", img.shape)
plt.figure(figsize=(7, 5))
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")
plt.show()
```

Choose Files bill.jpg

bill.jpg(image/jpeg) - 15207 bytes, last modified: 9/16/2026 - 100% done
Saving bill.jpg to bill.jpg
Image loaded successfully: bill.jpg
Image shape: (640, 479, 3)

Original Image



```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
# Mild Gaussian blur for noise reduction
gray = cv2.GaussianBlur(gray, (3, 3), 0)
plt.figure(figsize=(7, 5))
plt.imshow(gray, cmap="gray")
plt.title("Grayscale Preprocessed Image")
plt.axis("off")
plt.show()
```

Grayscale Preprocessed Image



```
img_keypoints = cv2.drawKeypoints(
    img,
    keypoints,
    None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)

plt.figure(figsize=(9, 6))
plt.imshow(cv2.cvtColor(img_keypoints, cv2.COLOR_BGR2RGB))
plt.title(f"SIFT Keypoints (n={len(keypoints)})")
plt.axis("off")
plt.show()

# Keypoint spread
if keypoints:
    xs = [kp.pt[0] for kp in keypoints]
    ys = [kp.pt[1] for kp in keypoints]

    print(f"Keypoint spread - X: [{min(xs):.1f}, {max(xs):.1f}]")
    print(f"Keypoint spread - Y: [{min(ys):.1f}, {max(ys):.1f}]")
```

SIFT Keypoints (n=99)



```
Keypoint spread - X: [23.9, 463.6]
Keypoint spread - Y: [82.6, 521.9]
```

```

hog_resized = cv2.resize(gray, (128, 128))

t0 = time.perf_counter()

hog_features, hog_image = hog(
    hog_resized,
    orientations=9,
    pixels_per_cell=(8, 8),
    cells_per_block=(2, 2),
    block_norm="L2-Hys",
    visualize=True
)

t_hog = time.perf_counter() - t0

print("HOG Feature Extraction")
print("-----")
print("HOG feature vector length:", hog_features.shape[0])
print(f"Time taken: {t_hog:.4f} seconds")

```

```

HOG Feature Extraction
-----
HOG feature vector length: 8100
Time taken: 0.2187 seconds

```

```

hog_image_rescaled = exposure.rescale_intensity(
    hog_image,
    in_range=(0, 10)
)

plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.imshow(hog_resized, cmap="gray")
plt.title("Input Image (Resized)")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(hog_image_rescaled, cmap="gray")
plt.title("HOG Descriptor")
plt.axis("off")

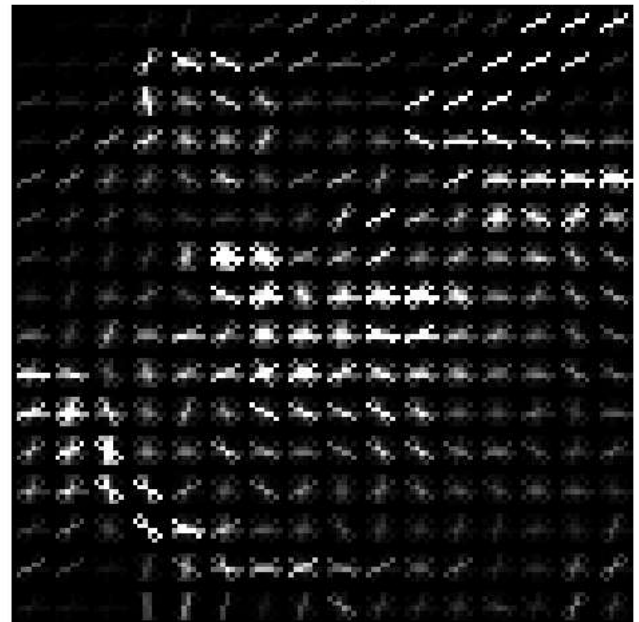
plt.tight_layout()
plt.show()

```

Input Image (Resized)



HOG Descriptor



```

print("SIFT vs HOG Comparison")
print("-----")

```

```

print(
    f"{'Property':25s} | "
    f"{'SIFT':30s} | "
    f"{'HOG':30s}"
)

print("-" * 95)

print(
    f"{'Type':25s} | "
    f"{'Sparse keypoints + descriptor':30s} | "
    f"{'Dense feature descriptor':30s}"
)

print(
    f"{'Scale invariance':25s} | "
    f"{'Yes':30s} | "
    f"{'No':30s}"
)

print(
    f"{'Rotation invariance':25s} | "
    f"{'Yes':30s} | "
    f"{'No':30s}"
)

print(
    f"{'Output size':25s} | "
    f"{'f'{len(keypoints)} x 128':30s} | "
    f"{'f'{hog_features.shape[0]} values':30s}"
)

print(
    f"{'Compute time (s)':25s} | "
    f"{'t_sift:<30.4f} | "
    f"{'t_hog:<30.4f}"
)

```

SIFT vs HOG Comparison

Property	SIFT	HOG
Type	Sparse keypoints + descriptor	Dense feature descriptor
Scale invariance	Yes	No
Rotation invariance	Yes	No
Output size	99 x 128	8100 values
Compute time (s)	0.1441	0.2187

```

# Create a similar image by rotating and scaling
# the original image

```

```

center = (
    gray.shape[1] // 2,
    gray.shape[0] // 2
)

```

```

rotation_matrix = cv2.getRotationMatrix2D(
    center,
    15,
    0.9
)

```

```

gray2 = cv2.warpAffine(
    gray,
    rotation_matrix,
    (gray.shape[1], gray.shape[0])
)

```

```

# Detect SIFT features in second image
kp2, des2 = sift.detectAndCompute(gray2, None)

```

```

print("Image 2 keypoints:", len(kp2))

```

```

# Match descriptors
if descriptors is not None and des2 is not None:

```

```

bf = cv2.BFMatcher(
    cv2.NORM_L2,
    crossCheck=True
)

matches = bf.match(
    descriptors,
    des2
)

matches = sorted(
    matches,
    key=lambda m: m.distance
)

print("Total matches found:", len(matches))

if matches:
    print(
        f"Best match distance: "
        f"{matches[0].distance:.2f}"
    )

    # Draw top 30 matches
    match_img = cv2.drawMatches(
        gray,
        keypoints,
        gray2,
        kp2,
        matches[:30],
        None,
        flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
    )

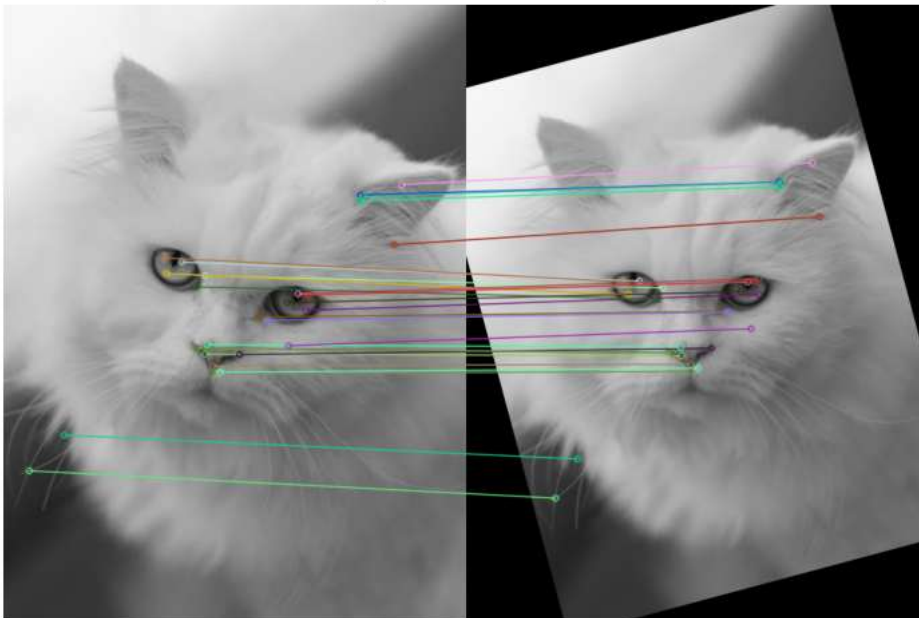
    plt.figure(figsize=(14, 6))
    plt.imshow(match_img, cmap="gray")
    plt.title("Top 30 SIFT Matches")
    plt.axis("off")
    plt.show()

else:
    print("SIFT descriptors could not be generated.")

```

Image 2 keypoints: 96
 Total matches found: 70
 Best match distance: 39.60

Top 30 SIFT Matches



```

if (
    descriptors is not None

```

```

descriptors is not None
and des2 is not None
and len(matches) > 0
):

    good_matches = [
        m for m in matches
        if m.distance < 200
    ]

    print("SIFT Matching Evaluation")
    print("-----")

    print("Total matches:", len(matches))

    print(
        "Good matches (distance < 200):",
        len(good_matches)
    )

    print("\nSIFT Strengths:")
    print("- Scale invariant")
    print("- Rotation invariant")
    print("- Robust to moderate illumination changes")
    print("- Useful for image matching and object recognition")

    print("\nSIFT Limitations:")
    print("- Computationally more expensive")
    print("- Produces many keypoints")
    print("- Can be slower for large images")

    print("\nHOG Strengths:")
    print("- Captures shape and edge information")
    print("- Simple and interpretable")
    print("- Useful for object detection")

    print("\nHOG Limitations:")
    print("- Not inherently scale invariant")
    print("- Not inherently rotation invariant")
    print("- Works best with fixed-size images")

```

SIFT Matching Evaluation

Total matches: 70

Good matches (distance < 200): 62

SIFT Strengths:

- Scale invariant
- Rotation invariant
- Robust to moderate illumination changes
- Useful for image matching and object recognition

SIFT Limitations:

- Computationally more expensive
- Produces many keypoints
- Can be slower for large images

HOG Strengths:

- Captures shape and edge information
- Simple and interpretable
- Useful for object detection

HOG Limitations:

- Not inherently scale invariant
- Not inherently rotation invariant
- Works best with fixed-size images